

Paradigma Orientado a Objetos

Paradigmas de Programación

Un paradigma de programación representa un enfoque particular o filosofía para el desarrollo de software, proporcionando conceptos a través de los cuales los elementos de un programa pueden ser representados y manipulados. No existe un paradigma que sea intrínsecamente mejor que otro, sino que su utilidad depende de la idoneidad para resolver un problema específico.

Programación Estructurada (Procedimental)

La programación estructurada o procedimental es un tipo de programación imperativa donde las instrucciones se organizan en procedimientos o rutinas que pueden ser invocadas según sea necesario. Este paradigma se caracteriza por un flujo de información clásico de entrada-proceso-salida, donde las aplicaciones tienen un único punto de entrada y de salida. En este modelo, el programa se ve como una secuencia de pasos lógicos donde los datos son pasivos y los procedimientos operan sobre ellos de manera externa. Los lenguajes puramente procedimentales, como C, carecen de conceptos como la herencia o los objetos.

Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos es un paradigma que organiza el software como una colección de objetos cooperativos, cada uno de los cuales es una instancia de una clase. A diferencia del modelo estructurado, la POO integra estrechamente los datos y los procedimientos que los manipulan, denominados métodos. Este enfoque permite modelar los problemas de una manera más cercana a cómo los seres humanos interactuamos con la realidad, viendo el mundo como un conjunto de entes rodeados de características y comportamientos. Java es un lenguaje multi-paradigma que, aunque es predominantemente orientado a objetos, hereda y utiliza conceptos de la programación procedimental.

Diferencias en Terminología y Flujo

La transición de la programación estructurada a la POO implica un cambio significativo en la terminología y la lógica de ejecución. Mientras que en la programación procedimental se habla de "procedimientos", en POO se denominan "métodos". Las estructuras de datos en el modelo procedimental se etiquetan como "registros", mientras que en el paradigma de objetos se convierten en "objetos" o "instancias". En cuanto al flujo, el modelo clásico es lineal y jerárquico, mientras que el modelo de objetos se basa en el envío de mensajes entre entidades independientes que colaboran para realizar tareas.

Conceptos Fundamentales: Clase, Objeto e Instancia

El paradigma de objetos se sustenta en tres conceptos que a menudo se confunden pero que tienen definiciones técnicas precisas en el análisis y diseño de software.

La Clase: El Plano de Construcción

Una clase es una descripción abstracta de un conjunto de objetos que comparten características similares. Se define como una plantilla, plano o prototipo que determina las variables (atributos) y las funciones (métodos) que serán comunes a todos los elementos de ese tipo. Técnicamente, una clase es un tipo de dato definido por el usuario que establece la anatomía de los objetos futuros. En Java, la definición de una clase comienza con la palabra reservada `class` y se guarda típicamente en un archivo con el mismo nombre y extensión `.java`. Por ejemplo, la clase `Coche` no es un coche real, sino la abstracción que define que todos los coches tienen una matrícula, marca y modelo.

El Objeto: La Entidad Tangible

Un objeto es una entidad tangible que exhibe un comportamiento bien definido y juega un papel específico dentro del dominio de un problema. Es el centro de la POO y representa la materialización de los conceptos definidos en la clase. A diferencia de la clase, que es un concepto abstracto, el objeto existe en la memoria de la computadora y posee una identidad propia que lo diferencia de otros objetos, incluso si pertenecen a la misma clase. Por ejemplo, "mi coche" y "el coche del vecino" son dos objetos distintos que comparten la estructura de la clase `Coche`.

La Instancia: El Acto de Creación

El término instancia se utiliza a menudo indistintamente con "objeto", pero técnicamente se refiere a la manifestación concreta de una clase en un momento dado. El proceso de crear un objeto a partir de una clase se denomina "instanciación". En este momento, el sistema operativo reserva memoria suficiente para almacenar todos los atributos definidos por la clase para ese ejemplar específico. En Java, esto se logra mediante el operador `new`, que invoca al constructor de la clase para dar vida al nuevo objeto.

Anatomía de un Objeto: Identidad, Estado y Comportamiento

Para que un elemento sea considerado un objeto dentro de este paradigma, debe poseer tres componentes esenciales: identidad, estado y comportamiento.

Identidad

La identidad es la característica que permite diferenciar un objeto de cualquier otro. En términos de programación, es el nombre único o identificador que el desarrollador asigna

a un objeto al crearlo (por ejemplo: `alumno1`, `cuentaCorriente`). Esta identidad permite que el sistema rastree cada ejemplar de forma independiente en la memoria del programa.

Atributos y Estado

Los atributos son el conjunto de rasgos, propiedades o características que posee un objeto y que lo definen dentro del contexto de una aplicación. Estos se almacenan en variables, también llamadas variables de instancia. El **estado** de un objeto se refiere al valor o conjunto de valores que toman sus atributos en un determinado momento de la ejecución. Dado que los valores de las variables pueden cambiar a lo largo del tiempo (por ejemplo, el saldo de una cuenta bancaria), el estado es una fotografía dinámica de la información del objeto.

Métodos y Comportamiento

El comportamiento está determinado por los métodos, que son el conjunto de acciones o funciones que el objeto puede realizar. Los métodos definen cómo interactúa el objeto con su entorno y con otros objetos. Representan las operaciones permitidas sobre los datos del objeto. Por ejemplo, un objeto `Gato` puede tener métodos como `maullar()`, `comer()` o `dormir()`, los cuales especifican su conducta dentro del sistema.

Modelado del Mundo Real mediante Objetos

El atractivo intuitivo de la POO reside en su capacidad para representar el mundo real de la manera más fiel posible mediante el proceso de modelado.

El Proceso de Abstracción

La abstracción es un pilar fundamental que consiste en identificar y aislar las características esenciales de un objeto para un problema específico, ignorando los detalles no relevantes. No se trata de copiar la realidad completa, sino de simplificarla para manejar la complejidad. Por ejemplo, si modelamos un `Estudiante` para un sistema de becas, el atributo `promedio` es esencial, mientras que el `color de ojos` es irrelevante y debe omitirse mediante la abstracción. Una buena abstracción permite que las cosas complejas se vean simples y genera una dependencia mínima entre los componentes del sistema.

Identificación de Entes, Rasgos y Acciones

Para realizar un modelado efectivo, el programador debe analizar el problema y hacerse preguntas clave: ¿Qué entes están involucrados? (Objetos), ¿Qué características comunes tienen? (Atributos) y ¿Qué acciones realizan? (Métodos). En un sistema para una liga de baloncesto, los objetos modelados serían `Jugador`, `Partido` y `Estadio`. Un `Coche` en el mundo real se traduce al software conservando sus rasgos significativos (marca, modelo) y sus acciones asociadas (acelerar, frenar).

Del Análisis a la Implementación

El modelado sigue un ciclo de vida que comienza con el **Análisis**, donde se define el problema y los datos requeridos. Continúa con el **Diseño**, donde se plantea el modelo de solución y se definen las estructuras de las clases (a menudo usando diagramas UML). Finalmente, la **Implementación** traduce ese diseño a un lenguaje de programación específico como Java. Este enfoque modular facilita que el software sea flexible, mantenible y capaz de evolucionar ante cambios futuros.

Definición de Clases en Java: Atributos, Métodos y Constructores

La declaración de una clase es el primer paso práctico para implementar el diseño orientado a objetos.

Definición de Atributos

Los atributos representan los datos de los objetos y se controlan mediante la declaración de variables dentro del cuerpo de la clase. Existen dos tipos principales de variables en este contexto:

1. **Variables de instancia:** Se crea una copia separada de estas variables para cada objeto instanciado. Cada objeto tiene sus propios valores independientes.
2. **Variables de clase (estáticas):** Se declaran con la palabra `static`. Solo existe una copia de la variable para todos los objetos de la clase. Si un objeto modifica su valor, el cambio afecta a todos los demás. Son útiles para constantes (como π) o contadores globales.

Ejemplo en Java:

```
public class Estudiante {  
    // Atributos de instancia  
    private String nombre;  
    private double promedio;  
  
    // Atributo de clase (estático)  
    public static String nombreInstitucion = "TecNM";  
}
```

Definición de Métodos

Los métodos son funciones que operan sobre los datos de la clase. Al declarar un método, es obligatorio especificar el tipo de dato que devuelve (o `void` si no devuelve nada), su nombre y una lista de parámetros entre paréntesis. Los parámetros son

variables locales que solo existen dentro del método para recibir información externa necesaria para la operación. Dentro de los métodos, se puede usar la palabra reservada `this` para referenciar explícitamente a los atributos del objeto actual, especialmente cuando los parámetros tienen el mismo nombre que los atributos (ocultación).

El Método Constructor

El constructor es un método especial cuya misión es inicializar un objeto de una clase al momento de su creación. Posee reglas estrictas: debe tener el mismo nombre exacto que la clase y no puede devolver ningún valor, ni siquiera `void`. Se ejecuta automáticamente cuando se utiliza el operador `new`.

Existen varios tipos de constructores que ofrecen flexibilidad al diseño:

- **Constructor por defecto:** Si el programador no escribe ningún constructor, Java proporciona uno automáticamente que no recibe parámetros e inicializa los atributos a valores nulos o cero.
- **Constructor con parámetros:** Permite asignar valores específicos a los atributos desde el momento de la instanciación.
- **Sobrecarga de constructores:** Consiste en definir múltiples constructores en la misma clase que se diferencian por el número, orden o tipo de sus parámetros. El compilador decide cuál ejecutar basándose en los argumentos proporcionados al crear el objeto.

Ejemplo de constructores:

```
public class Punto {  
    private int x, y;  
  
    // Constructor vacío (por defecto explícito)  
    public Punto() {  
        this.x = 0;  
        this.y = 0;  
    }  
  
    // Constructor con parámetros (Sobrecarga)  
    public Punto(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

Encapsulación y Modificadores de Acceso

La encapsulación es el proceso de agrupar datos y operaciones relacionadas bajo una misma unidad, protegiendo la información de interferencias externas.

El Concepto de Caja Negra

Básicamente, la encapsulación trata a la clase como una "caja negra" donde se ocultan los detalles internos de implementación. Los usuarios del objeto solo necesitan conocer su interfaz pública (lo que hace) y no cómo está programado internamente. Esto permite cambiar la lógica interna de una clase sin afectar a otras partes del programa que la utilizan.

Modificadores de Acceso en Java

Java utiliza modificadores para definir el ámbito de visibilidad de los miembros de una clase. Los niveles de acceso son:

- **private:** Es el más restrictivo. El miembro solo es visible dentro de la misma clase. Se recomienda para todos los atributos para garantizar la integridad de los datos.
- **public:** El miembro es accesible desde cualquier lugar. Se utiliza para los métodos que forman la interfaz pública.
- **protected:** Permite el acceso desde la misma clase, clases del mismo paquete y subclases, incluso en otros paquetes.
- **Acceso por defecto (paquete):** Si no se especifica modificador, el acceso es limitado a clases dentro del mismo paquete.

Métodos de Acceso: Getters y Setters

Dado que los atributos suelen ser privados, se utilizan métodos públicos denominados **getters** y **setters** para interactuar con ellos de forma controlada.

- **Getter (Consultor):** Su cometido es devolver el valor de un atributo privado. Sigue la convención `getNombreAtributo()` o `isNombreAtributo()` para booleanos.
- **Setter (Modificador):** Permite establecer o cambiar el valor de un atributo. Su gran ventaja es que permite incluir lógica de **validación** para evitar que el objeto alcance un estado inconsistente (por ejemplo, impedir que un precio sea negativo).

Ejemplo de encapsulamiento:

```
public class CuentaBancaria {  
    private double saldo; // Atributo protegido  
  
    // Getter  
    public double getSaldo() {  
        return this.saldo;  
    }  
}
```

```
// Setter con validación
public void setSaldo(double saldo) {
    if (saldo >= 0) {
        this.saldo = saldo;
    } else {
        System.out.println("Error: El saldo no puede ser negativo");
    }
}
}
```

Introducción al Diseño de Clases Simples

El diseño de clases efectivas requiere un equilibrio entre la funcionalidad y la organización.

Cohesión y Acoplamiento

Un diseño de calidad busca una **fuerte cohesión**, lo que significa que cada clase debe ser responsable de una única tarea o concepto bien definido (Principio de Responsabilidad Única). Al mismo tiempo, se busca un **bajo acoplamiento**, minimizando las dependencias directas entre clases para que los cambios en una no rompan el funcionamiento de las demás.

Representación en UML

El Lenguaje de Modelado Unificado (UML) permite visualizar las clases antes de codificarlas. Gráficamente, una clase se representa con un rectángulo dividido en tres secciones: el nombre de la clase en la parte superior, los atributos en el centro y los métodos u operaciones en la parte inferior. Los signos + (public) y - (private) suelen anteceder a los miembros para indicar su visibilidad.

Comunicación mediante Mensajes

En un sistema orientado a objetos, el trabajo se realiza mediante el envío de mensajes. Un mensaje consta de la identidad del receptor, el método que debe ejecutar y cualquier información adicional necesaria (argumentos). En Java, esto se traduce sintácticamente como `objeto.metodo(parametros)`.

Tipos de Datos y Estructuras Aplicadas a Objetos

La POO maneja tanto tipos de datos elementales como estructuras complejas que permiten organizar colecciones de objetos.

Tipos Primitivos y Tipos Objeto

En Java coexisten los tipos de datos primitivos (como `int`, `double`, `char`, `boolean`) y los tipos objeto. Es fundamental identificar el tipo correcto para cada atributo durante la fase de abstracción para asegurar que los datos se manipulen eficientemente.

Arrays y Colecciones de Objetos

Cuando es necesario manejar múltiples objetos del mismo tipo, se recurre a estructuras como los **Arrays de objetos** o listas dinámicas. Un array de objetos funciona como una colección de elementos donde cada posición almacena una referencia a una instancia de una clase. Estructuras más avanzadas, como `ArrayList`, permiten gestionar colecciones cuyo tamaño puede crecer o disminuir dinámicamente durante la ejecución del programa.

Relaciones de Composición ("Tiene-un")

La composición es una forma poderosa de reutilización de software donde una clase contiene referencias a objetos de otras clases como miembros. Refleja una relación jerárquica de "parte-de" o "tiene-un". Por ejemplo, una clase `Persona` puede tener un atributo de tipo `Fecha` para representar su nacimiento. Esto permite construir sistemas complejos ensamblando componentes más simples y autónomos.